Connecting Claude Code to TAG Marketing Client Websites

Standard Operating Procedure • Author: Kim Bandola • Date: 21 May 2026

DRAFT — for review

Purpose of this document

This document explains, in plain language, how Claude Code on Kim's laptop is connected to TAG Marketing's three plumbing-industry WordPress websites — Summit Plumbing Pros, Pipeliners, and Alphalete Trenchless Pipelining — and how she uses Claude to work on them day-to-day. It is intended as a reference for Kim, and as an overview for anyone (such as Scott) who needs to understand the setup without reading any code.

The three websites in scope

Each website is a WordPress installation belonging to one of TAG Marketing's plumbing-industry clients. The three sites live on two different hosting providers, but the day-to-day workflow Kim uses to interact with them is the same.

They are:

- Summit Plumbing Pros — hosted at SiteGround.
- Pipeliners — hosted at Hostinger.
- Alphalete Trenchless Pipelining — also hosted at Hostinger.

For each site, two separate access paths exist: a browser-based path (WordPress Admin) and a behind-the-scenes path (a secure command-line connection called SSH). Claude uses the second path only when a task genuinely requires it.

What "connected" actually means

When Kim says Claude is "connected" to a website, it really means three things have been put in place:

- Kim's laptop holds a small private digital key that proves her identity to the server. This key never leaves the laptop.
- A matching public key has been uploaded to the website's server by Scott. The public half is safe to share — it can only be used to verify the private half, not impersonate it.
- Kim's laptop knows the server's address, port number, and which key to use, all written into a small text file called the SSH configuration. Once that file is set, she can refer to a site by a short nickname ("summit", "pipeliners", "alphalete") and her computer fills in the rest.

Claude Code, running on Kim's laptop, uses exactly the same SSH configuration. So when Kim asks Claude to look at a server, Claude connects under Kim's identity, not its own.

How Kim works with these sites day-to-day

Most work does not require touching the server directly. The vast majority of tasks happen in two places:

- WordPress Admin — the standard web dashboard, opened in a browser. Content updates, page edits, plugin settings, and most everyday work happens here.
- The doe-* Python scripts — internal tooling that talks to each site's API. This is how Kim runs reports, bulk operations, and anything she wants to script.

The SSH connection is the last-resort tool. It only gets used when something needs investigating at the server level — checking error logs, running a database query, or installing a plugin by hand. When a task like that comes up, Kim simply tells Claude "connect to summit and show me the latest error log," and Claude uses the SSH alias to do it.

Setting up a new website (overview)

The setup process for each new site follows the same pattern. It takes Kim about ten minutes; the only outside dependency is Scott, who has to upload the key on the server side.

The five-step shape

- Generate a brand-new digital key on the laptop, dedicated to this one site.
- Print the public half of the key and send it to Scott in a message.
- Wait for Scott to upload it to the host (SiteGround Site Tools, or Hostinger hPanel).
- Add a short configuration block to the laptop's SSH config file, giving the site a memorable nickname.
- Run a quick test to confirm the connection works.

Once those five steps are done, the nickname is permanent — Kim never has to repeat the setup unless the laptop is replaced or Scott removes the key. From that point on, Claude can connect to the site at any time.

Daily reminders

- Default to WordPress Admin and the doe-* scripts. SSH is reserved for the rare cases where the browser and scripts can't do the job.
- Never send the private key file to anyone — only the file ending in .pub is safe to share.
- If a site suddenly stops connecting, the cause is almost always one of: Scott hasn't (re-)uploaded the key, the hosting provider has changed Kim's allowed IP address, or the private key's file permissions have drifted.

When something stops working

If a connection that used to work suddenly refuses, walk through three checks before escalating:

- Confirm the key is still in place on the server. Ask Scott to look in Site Tools (SiteGround) or hPanel > SSH Access (Hostinger).

- Confirm Kim's current internet address is still allowed. Hostinger in particular sometimes restricts SSH to specific IPs; if the home IP changes, Scott may need to whitelist the new one.
- Confirm the local key files have not had their permissions changed. Windows occasionally re-permissions files in ways that SSH refuses. The fix is a one-line command in Git Bash.

If all three are healthy and the connection still fails, capture the verbose connection log and send it to Scott — it will tell him exactly which step of the handshake is failing.

Reference — the three sites at a glance

All technical details for the three current sites in one table:

Client	Nickname	Host	Port
Summit Plumbing Pros	summit	gcam1044.siteground.biz	18765
Pipeliners	pipeliners	77.37.58.118	65002
Alphalete Trenchless	alphalete	217.196.54.224	65002

The matching key files on Kim's laptop are siteground_summit, hostinger_pipeliners, and hostinger_alphalete. They live in the hidden .ssh folder of her user profile.

Technical appendix (for reference only)

The information below is included only so this document is self-contained. Day-to-day, the SOP "connect-website.md" file in the project root has the step-by-step commands.

Configuration block format

Each site gets a block like the one below in ~/.ssh/config on Kim's laptop. The values come from the table on the previous page.

```
Host <nickname>
    HostName <host>
    User <username from Scott>
    Port <port>
    IdentityFile ~/.ssh/<keyfile>
    IdentitiesOnly yes
    ServerAliveInterval 60
```

Connection test

After Scott uploads the key, this single command proves end-to-end success:

```
ssh -o ConnectTimeout=10 <nickname> "hostname && whoami"
```

Key generation

New keys are generated with the modern ed25519 algorithm:

```
ssh-keygen -t ed25519 -f ~/.ssh/<keyfile> -N "" -C "<comment>"
```

End of document. This is a DRAFT — please send feedback before treating it as final.